

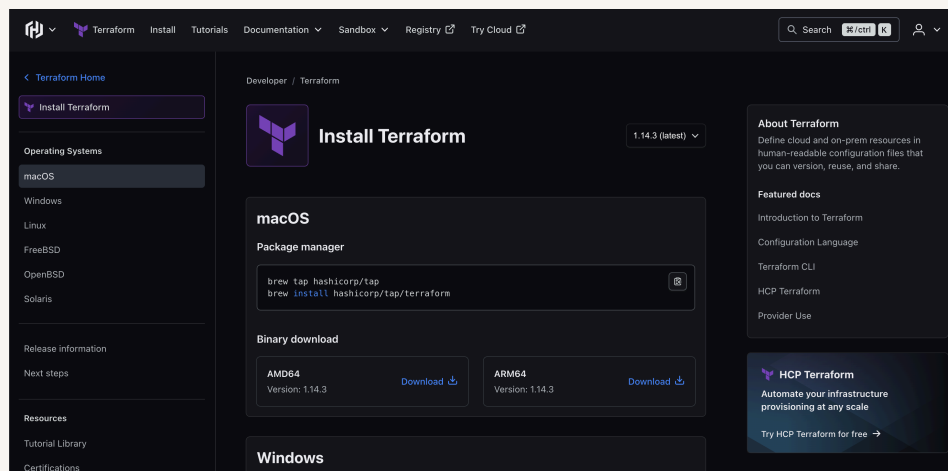


nextwork.org

Create S3 Buckets with Terraform

CH

Arpita Chowdhury





Introducing Today's Project!

In this project, I will demonstrate how to use Terraform to automate the creation and management of an AWS S3 bucket, including configuring AWS credentials, defining infrastructure as code, and uploading files to S3 using Terraform. The goal is to replace manual cloud setup with repeatable, error-free automation, showing how infrastructure can be provisioned, updated, and managed consistently just like real-world DevOps, DevSecOps, and Platform Engineering teams do.

Tools and concepts

Services I used were AWS S3, AWS IAM, AWS CLI, and Terraform. Key concepts I learnt include infrastructure as code (IaC) defining and managing cloud resources through version controlled configuration files along with Terraform's workflow (init, plan, apply), provider configuration, resource dependencies, secure credential management, tagging for governance, and managing both infrastructure (S3 buckets) and data (S3 objects) declaratively and repeatably.

Project reflection

This project took me approximately 2 hours to complete. The most challenging part was troubleshooting Terraform errors related to AWS credentials and configuration syntax, especially understanding how small mistakes can prevent Terraform from planning or applying changes. It was most rewarding to successfully automate the creation of an S3 bucket and upload a file using Terraform, seeing infrastructure and data managed entirely through code.

I chose to do this project today because I wanted hands-on experience with infrastructure as code and to practice automating AWS resources the same way real DevOps and platform teams do, instead of relying on manual setup in the console. Something that would make learning with NextWork even better is clearer upfront guidance on common beginner issues such as AWS credential

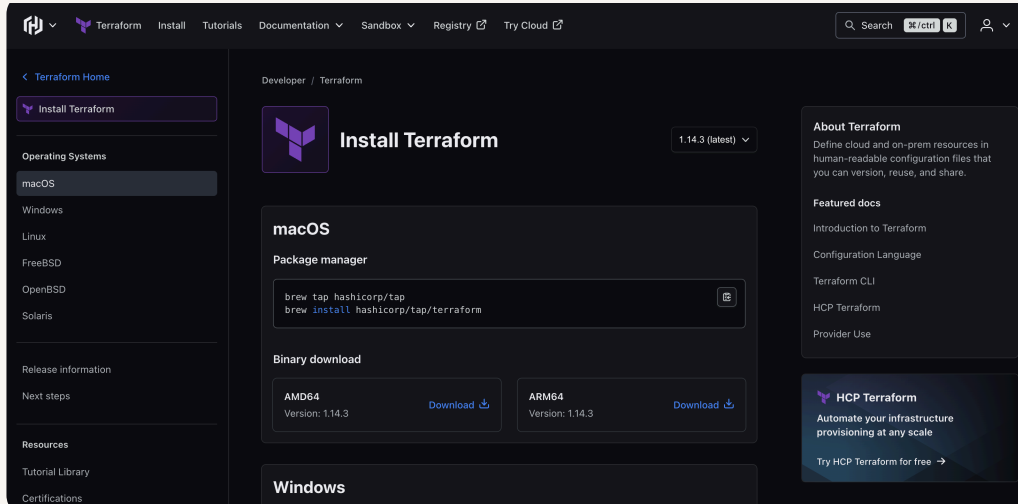


Introducing Terraform

Terraform is an infrastructure-as-code (IaC) tool that allows you to define, provision, and manage cloud resources using configuration files, so infrastructure can be created and updated automatically instead of manually through the console.

Terraform is one of the most popular tools used for infrastructure as code (IaC), which is the practice of defining and managing cloud infrastructure using version-controlled configuration files instead of manual setup, enabling automation, consistency, and repeatable deployments.

Terraform uses configuration files to describe what infrastructure should be created and how it should be managed in a declarative way. The `main.tf` file is the primary Terraform configuration file where resource definitions, providers, and settings are written to tell Terraform what to build.





Configuration files

The configuration is structured in logical blocks a provider block to define the cloud platform and region, followed by resource blocks to declare the S3 bucket and its public access settings. The advantage of doing this is that Terraform can clearly understand dependencies, apply security best practices by default, and manage the entire setup declaratively, making the infrastructure reproducible, secure, and easy to update or destroy.

My main.tf configuration has three blocks

The first block indicates the AWS provider configuration, specifying which cloud provider Terraform should use and which region (us-east-1) the resources will be created in. The second block provisions an Amazon S3 bucket, defining a globally unique bucket name and telling Terraform to create and manage that storage resource in AWS. The third block manages public access controls for the S3 bucket, explicitly blocking public ACLs and policies to ensure the bucket remains private and secure by default.



```
1 provider "aws" {
2   region = "us-east-1"
3 }
4
5 resource "aws_s3_bucket" "my_bucket" {
6   bucket = "nextwork-unique-bucket-Arpita-10" # Make sure this bucket name is globally unique by typing a long random number
7 }
8
9 resource "aws_s3_bucket_public_access_block" "my_bucket_public_access_block" {
10  bucket = aws_s3_bucket.my_bucket.id
11
12  block_public_acls       = true
13  ignore_public_acls     = true
14  block_public_policy     = true
15  restrict_public_buckets = true
16 }
17
```



Customizing my S3 Bucket

For my project extension, I visited the official Terraform Registry to understand how to correctly define and customize AWS resources using Terraform, specifically the `aws_s3_bucket` resource. The documentation shows the required and optional arguments for creating an S3 bucket, examples of real-world configurations, security best practices (such as blocking public access), and how different S3-related resources work together, which helped me confidently extend my configuration without guessing or trial-and-error.

I chose to customise my bucket by adding tags such as Project, Owner, and Environment using Terraform, because tagging helps organize cloud resources, track ownership and costs, and follow real-world infrastructure best practices. When I launch my bucket, I can verify my customization by viewing the tags attached to the S3 bucket in the AWS Management Console or by checking the resource details after Terraform applies the configuration.



```
main.tf
Users > arpitachowdhury > Desktop > nextwork_terraform > main.tf
1 provider "aws" {
2   region = "us-east-1"
3 }
4
5 resource "aws_s3_bucket" "my_bucket" {
6   bucket = "nextwork-unique-bucket-arpita-10"
7
8   tags = {
9     Project = "NextWork-Terraform"
10    Owner   = "Arpita"
11    Environment = "Dev"
12  }
13 }
14
15 resource "aws_s3_bucket_public_access_block" "my_bucket_public_access_block" {
16   bucket = aws_s3_bucket.my_bucket.id
17
18   block_public_acls       = true
19   ignore_public_acls     = true
20   block_public_policy     = true
21   restrict_public_buckets = true
22 }
23 |
```



Terraform commands

I ran `terraform init` to initialize my Terraform project, which set up the working directory by downloading the required AWS provider plugin, configuring the backend, and creating the `.terraform.lock.hcl` file so Terraform can consistently use the same provider versions in future runs.

Next, I ran `terraform plan` to preview the changes Terraform would make to my AWS environment, allowing me to see exactly which resources (like the S3 bucket and its settings) would be created or modified before actually applying them, so I could catch mistakes and confirm everything looked correct.



```
nextwork_terraform — -zsh — 80x24
arpitachowdhury@Arpitas-MacBook-Air nextwork_terraform % touch main.tf
arpitachowdhury@Arpitas-MacBook-Air nextwork_terraform % terraform init

Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.27.0...
- Installed hashicorp/aws v6.27.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
arpitachowdhury@Arpitas-MacBook-Air nextwork_terraform %
```



AWS CLI and Access Keys

When I tried to plan my Terraform configuration, I received an error message that says “No valid credential sources found” (and “failed to refresh cached credentials... no EC2 IMDS role found”) because Terraform’s AWS provider couldn’t find any AWS credentials configured on my laptop yet. Since I’m running Terraform locally (not on an EC2 instance), there is no instance role/metadata service to pull credentials from—so I need to create AWS access keys and configure the AWS CLI (or set environment credentials) so Terraform can authenticate to AWS.

To resolve my error, first I installed AWS CLI, which is a command-line tool that lets me securely interact with AWS services from my terminal. It allows me to authenticate using access keys, manage resources like S3 and EC2, and provides the credentials Terraform needs to communicate with AWS on my behalf.

I set up AWS access keys to authenticate my local machine with my AWS account so tools like the AWS CLI and Terraform can securely create, read, and manage AWS resources on my behalf. The access key ID and secret access key act as credentials that prove my identity and permissions when Terraform runs commands such as planning and provisioning infrastructure.



```
arpitachowdhury@Arpitas-MacBook-Air nextwork_terraform % terraform plan

Planning failed. Terraform encountered an error while generating this plan.

Error: No valid credential sources found

  with provider["registry.terraform.io/hashicorp/aws"],
  on main.tf line 1, in provider "aws":
   1: provider "aws" {

Please see https://registry.terraform.io/providers/hashicorp/aws
for more information about providing credentials.

Error: failed to refresh cached credentials, no EC2 IMDS role found,
operation error ec2imds: GetMetadata, request canceled, context deadline
exceeded

arpitachowdhury@Arpitas-MacBook-Air nextwork_terraform % curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
sudo installer -pkg AWSCLIV2.pkg -target /

% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100 46.8M  100 46.8M    0     0  17.0M      0  0:00:02  0:00:02 --:--:-- 17.0M
Password:
installer: Package name is AWS Command Line Interface
installer: Installing at base path /
installer: The install was successful.
arpitachowdhury@Arpitas-MacBook-Air nextwork_terraform % aws --version

aws-cli/2.32.28 Python/3.13.11 Darwin/24.6.0 exe/arm64
arpitachowdhury@Arpitas-MacBook-Air nextwork_terraform %
```




Lanching the S3 Bucket

I ran terraform apply to apply my Terraform configuration and actually create the S3 bucket defined in main.tf. Running terraform apply will affect my AWS account by provisioning real AWS resources (like an S3 bucket) exactly as described in the configuration, making permanent changes unless they are later modified or destroyed using Terraform.

The sequence of running terraform init, terraform plan, and terraform apply is crucial because each step builds on the previous one: init prepares the environment and downloads required providers, plan safely previews the changes Terraform will make, and apply executes those changes. Skipping or reordering these steps can lead to errors, missing dependencies, or unintended changes to the AWS environment.



General purpose buckets

All AWS Regions

Directory buckets

General purpose buckets (2) Info

Copy ARN

Empty

Delete

Create bucket

Buckets are containers for data stored in S3.

Find buckets by name

< 1 > ⚙

	Name ▲	AWS Region ▼	Creation date ▼
☐	elasticbeanstalk-us-east-1-006834263464	US East (N. Virginia) us-east-1	January 2, 2026, 14:45:31 (UTC-06:00)
☐	nextwork-unique-bucket-arpita-10	US East (N. Virginia) us-east-1	January 3, 2026, 01:05:33 (UTC-06:00)



Uploading an S3 Object

We need to run terraform apply again because we updated the Terraform configuration by adding a new resource (`aws_s3_object`), and applying the configuration is required to make Terraform create or update real resources in AWS based on those new changes.

To validate that I've updated my configuration successfully, I checked the Terraform apply output for a successful completion and verified that the file appeared inside my S3 bucket in the AWS Management Console, confirming Terraform created the new S3 object as defined.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

